AUTOMIND

CONFIGURACIÓN MANUAL / GOOGLE CLOUD

React Native + TypeScript | PostgreSQL y cuatro tablas | 3 de noviembre de 2026

1. Qué incluye esta versión

Una sola base PostgreSQL llamada automind contiene exactamente cuatro tablas de la aplicación: usuarios, modelos, permisos y solicitudes. La API está escrita únicamente en Python, con FastAPI, SQLAlchemy, pg8000 y el conector oficial de Cloud SQL. Utiliza consultas SQL parametrizadas. TypeScript se utiliza en la app móvil. La app móvil conserva React Native, TypeScript, Skia, RaTeX y login únicamente Google Gmail.

Los 24 modelos siguen siendo calculadoras: Input 1 + Input 2 + Input 3. Admiten negativos y decimales, con rango de -1.000.000.000.000 a 1.000.000.000.000 por input. Sus IDs son enteros únicos de ocho dígitos, del 10000001 al 10000024. Los nombres KS, CART, Red Neuronal, Regresión, Holt Winters y Prophet son etiquetas; todos suman.

Los archivos están organizados primero por modelos y luego por área: inference/modelos/inmobiliario/cart.py, por ejemplo. Cada Python declara MODEL_ID y contiene su propia función calcular. En la tabla modelos, la columna archivo guarda la ruta relativa desde la raíz del repositorio privado de GitHub.

GitHub aporta las fuentes durante el build. Cloud Run ejecuta la copia del código incluida en su contenedor. El ID se resuelve mediante un registro fijo de módulos permitidos; cambiar la columna archivo por sí sola no ejecuta otro archivo. La app no envía rutas ni credenciales de GitHub.

El archivo sql/02-catalogo-y-accesos.sql prepara 24 fichas y 12 permisos para artemioaday@gmail.com: KS, CART y Red Neuronal por área. Tu registro de usuario se crea al primer login Google real. Los permisos previos quedan reservados por Gmail y se vinculan al subject verificado al registrarte.

Toda la configuración de nube se realiza con interfaces: Google Cloud Console, Cloud SQL Studio y GitHub web. En Studio pegarás SQL y pulsarás Ejecutar; no necesitas Cloud Shell. La sección Android utiliza herramientas locales para compilar.

2. Proyecto, APIs y cuentas de servicio

Abre https://console.cloud.google.com y selecciona o crea tu proyecto. Anota su ID y vincula una cuenta de facturación. En APIs y servicios > Biblioteca habilita Cloud Run Admin API, Cloud Build API, Artifact Registry API, Cloud SQL Admin API, Secret Manager API e Identity and Access Management API.

En IAM y administración > Cuentas de servicio crea automind-api. En acceso al proyecto asigna Cloud SQL Client (roles/cloudsql.client). Esta cuenta ejecuta la API y abre el canal seguro hacia PostgreSQL. No necesita Cloud Datastore User ni permisos de Firestore.

Crea automind-inference sin roles de base de datos. Ejecutará las calculadoras privadas. Crea automind-build para compilar y desplegar, con Cloud Build Service Account (roles/cloudbuild.builds.builder) y Cloud Run Admin (roles/run.admin).

Abre las cuentas automind-api y automind-inference > Permisos > Otorgar acceso. Concede a automind-build el rol Service Account User (roles/iam.serviceAccountUser) sobre cada una. No generes claves JSON para las cuentas; Cloud Run proporciona sus credenciales.

En el asistente de despliegue selecciona automind-build como cuenta de compilación. Si el trigger se crea con otra cuenta, abre Cloud Build > Triggers > Editar > Service account, selecciona automind-build, guarda y pulsa Ejecutar.

La cuenta administradora que configura la nube necesita permisos para crear instancias, usuarios SQL, secretos, servicios y roles. Para usar Studio, puede necesitar Cloud SQL Studio User (roles/cloudsql.studioUser), además de las credenciales del administrador PostgreSQL. Los Gmail de la app no reciben roles IAM.

3. Crear la instancia PostgreSQL

En Google Cloud abre SQL > Crear instancia > PostgreSQL. Usa ID de instancia automind-pg. Selecciona PostgreSQL 17 y edición Enterprise. Para comenzar, elige una configuración de desarrollo/prueba compatible con tu presupuesto y revisa la estimación del formulario antes de crearla.

Configura la contraseña del administrador postgres y guárdala de forma privada. Elige la región donde desplegarás la API, por ejemplo Santiago / southamerica-west1. Abre las opciones de conexiones: habilita Public IP. No agregues 0.0.0.0/0 ni redes autorizadas generales. La API se conectará mediante el conector oficial, con autorización IAM y cifrado.

La IP pública de esta guía se usa para el conector y no requiere exponer una conexión directa por IP al móvil. No configures certificados de cliente manuales ni un modo que los exija. Para utilizar únicamente IP privada sería necesaria conectividad VPC adicional; esa variante no es la configuración de este manual.

En copias de seguridad configura la política de respaldo y recuperación que necesites. Para una instalación inicial puedes usar una única zona; revisa la disponibilidad requerida antes de producción. Crea la instancia y espera su estado disponible.

En Información general copia el Connection name completo, con formato PROYECTO:REGIÓN:automind-pg. No es la dirección IP ni el nombre de la base. Lo usarás como INSTANCE_CONNECTION_NAME de la API.

Cloud SQL tiene cargos de instancia, capacidad y almacenamiento aunque la app no reciba solicitudes. Mínimo cero instancias en Cloud Run no apaga PostgreSQL. En Facturación crea un presupuesto y alertas; el aviso no detiene automáticamente el gasto. No se fijan precios en este documento.

4. Crear la base y las cuatro tablas

Dentro de la instancia abre Bases de datos > Crear base de datos. Escribe automind y confirma. En Usuarios > Agregar cuenta crea el usuario de base de datos automind_app con una contraseña diferente a postgres. Guárdala: se almacenará como secreto en el paso 5.

Abre Cloud SQL Studio en el menú de la instancia. Selecciona autenticación integrada de base de datos (Built-in), base automind, usuario postgres y su contraseña. Elige Nueva consulta. Abre localmente sql/01-esquema.sql, copia todo su contenido al editor y pulsa Ejecutar. Después ejecuta de la misma forma sql/02-catalogo-y-accesos.sql y, por último, sql/03-privilegios-app.sql.

El primer archivo crea las cuatro tablas con claves foráneas y restricciones. El segundo carga las 24 fichas con sus rutas y 12 permisos reservados para tu Gmail. No crea un usuario Google ficticio. El tercero limita automind_app: puede registrar identidades verificadas, gestionar solicitudes y cuota, pero no otorgar permisos ni editar el catálogo. No uses postgres como usuario de la API.

El tercer archivo retira la membresía cloudsqlsuperuser que Cloud SQL puede conceder al crear usuarios integrados. También quita privilegios de creación de bases/roles y asigna los permisos SQL concretos de la aplicación. Ejecútalo antes de desplegar la API.

En Studio comprueba las tablas con:

```
SELECT table_name FROM information_schema.tables
WHERE table_schema='public' AND table_type='BASE TABLE'
ORDER BY table_name;
```

Deben aparecer modelos, permisos, solicitudes y usuarios. El contador de cuota se guarda en usuarios: no hay una quinta tabla. Si ejecutas de nuevo los archivos iniciales, las tablas existentes y los datos ya presentes se conservan. No restauran automáticamente permisos que hayas deshabilitado.

5. Guardar la contraseña y preparar la conexión

Abre Secret Manager > Crear secreto. Nombre: automind-db-password. Como valor escribe únicamente la contraseña de automind_app, sin comillas y sin salto de línea adicional. Guarda el secreto y anota la versión, por ejemplo 1.

En los permisos de ese secreto agrega el email completo de automind-api@TU_PROYECTO.iam.gserviceaccount.com y el rol Secret Manager Secret Accessor (roles/secretmanager.secretAccessor). Limita el acceso al secreto necesario; el motor de calculadoras no necesita esta contraseña.

La API utiliza estas variables públicas de configuración y un secreto:

```
INSTANCE_CONNECTION_NAME = PROYECTO:REGIÓN:automind-pg
DB_NAME                  = automind
DB_USER                  = automind_app
GOOGLE_WEB_CLIENT_ID     = cliente Web OAuth real
INFERENCE_URL            = URL HTTPS del motor privado
DB_PASSWORD              = referencia al secreto, versión 1
```

DB_PASSWORD se configura en Cloud Run como Referencia a un secreto y Exponer como variable de entorno, no pegando la contraseña en el formulario de variables ordinarias. No va en GitHub, en mobile/src/config.ts ni en el PDF. La contraseña de postgres tampoco se utiliza en el contenedor.

El conector oficial usa la identidad de ejecución automind-api y su rol Cloud SQL Client para acceder a la instancia. El usuario y contraseña PostgreSQL determinan qué operaciones SQL permite esa conexión. Son controles distintos y ambos deben estar configurados.

La API mantiene como máximo cinco conexiones SQL por instancia de Cloud Run. Con máximo tres instancias habrá hasta quince conexiones de aplicación, además de las administrativas. El servidor valida la conexión al iniciar; un nombre de conexión, usuario o contraseña incorrecto puede impedir que la revisión arranque.

6. Configurar Google Sign-In

En Google Auth Platform abre Branding y completa AutoMind, email de soporte y datos requeridos. En Audience añade artemioaday@gmail.com a Test users durante las pruebas. En Data Access utiliza identidad básica: openid, email y profile. Esta app no requiere Firebase Authentication.

En Clients > Create client crea Web application. Guarda su Client ID terminado en .apps.googleusercontent.com. Ese mismo ID va en GOOGLE_WEB_CLIENT_ID de la API y googleWebClientId de mobile/src/config.ts. No se usa el secreto OAuth Web en el móvil.

Crea un cliente de tipo Android con package name com.automind y el SHA-1 del certificado que firmará la app. El paso 12 explica cómo obtenerlo con Android Studio. Registra los certificados debug y release que utilices; para Play App Signing añade también el certificado de firma mostrado en Play Console.

Para iOS crea un cliente con bundle ID com.automind. Copia su Client ID a googleIOSClientId y el iOS URL scheme a mobile/ios/AutoMind/Info.plist, reemplazando REEMPLAZAR_REVERSED_IOS_CLIENT_ID. El cliente Android, iOS y Web deben corresponder al proyecto adecuado.

La API verifica firma, audiencia, emisor, expiración y Gmail verificado. El registro usa el subject estable que acredita Google. Ninguna ruta acepta otro Gmail enviado en JSON para decidir permisos. El usuario autorizado puede consultar la Store y solicitar modelos; cada suma exige además su permiso específico.

7. Repositorio privado y calculadoras

En GitHub web crea o utiliza un repositorio privado. Con Add file > Upload files sube el contenido de la carpeta AutoMind extraída del ZIP. En la raíz deben quedar api, inference, sql, mobile, catalog y docs; evita una carpeta AutoMind adicional envolviendo todo. Confirma los cambios por lotes si la interfaz limita la cantidad de archivos.

No subas contraseñas, claves, node_modules, resultados build ni archivos .env privados. El ZIP contiene fuentes, SQL y archivos de configuración de ejemplo. El secreto de base de datos se crea desde la consola del paso 5.

Cada ID corresponde a un archivo independiente. Ejemplo:

```
inference/modelos/inmobiliario/cart.py
MODEL_ID = 10000002
```

En public.modelos, archivo contiene esa misma ruta relativa desde la raíz del repositorio. Dentro del contenedor, el Dockerfile de inference copia esa carpeta a /app; el módulo se importa mediante el registro fijo de IDs. No se descarga un Python desde GitHub al calcular.

Editar solo archivo en la tabla sirve para cambiar la referencia administrativa, no para ejecutar una ruta nueva. Para cambiar de ubicación un archivo debes moverlo en el repositorio, ajustar registry.py y la columna archivo de su modelo, comprobar MODEL_ID y volver a desplegar. Para cambiar la suma edita la función calcular del Python correspondiente.

La app muestra el ID y nombre; no necesita conocer las rutas del repositorio ni tener acceso a GitHub. La conexión de Cloud Build con el repositorio se autoriza en el siguiente paso.

8. Desplegar el motor privado

En Cloud Run pulsa Connect repository / Conectar repositorio. Selecciona Cloud Build, autentica GitHub y autoriza solo el repositorio elegido. Selecciona la rama main (expresión ^main$). En Build type elige Dockerfile y en Source location escribe inference/Dockerfile. Esa carpeta se usa como contexto de compilación.

Configura servicio automind-inference, región igual a la API y la base, cuenta de ejecución automind-inference, puerto 8080, 1 CPU, 512 MiB, mínimo 0 y máximo 3 instancias. En autenticación selecciona Requerir autenticación. La entrada de red All permite el canal de servicio, pero IAM seguirá rechazando llamadores sin autorización.

Selecciona automind-build como cuenta de compilación cuando se ofrezca el campo. Si el trigger utiliza otra cuenta, edítalo en Cloud Build > Triggers antes de volver a ejecutarlo. Guarda la configuración y crea el servicio. Abre el build y comprueba que las pruebas de los 24 Python pasan.

Espera el estado Ready del servicio. Copia la URL HTTPS sin barra final: será INFERENCE_URL de la API. El motor no recibe las variables DB_* ni el secreto PostgreSQL; solo ejecuta las calculadoras. No agregues allUsers a este servicio.

Los commits posteriores en la rama pueden iniciar automáticamente el trigger. Para lanzar un build desde la interfaz usa Cloud Build > Triggers > Ejecutar. Revisa el resultado del build y de la revisión antes de probar el cálculo.

9. Desplegar la API PostgreSQL

Crea otro servicio desde el mismo repositorio usando api/Dockerfile. Nombre automind-api, región igual a la base, cuenta de ejecución automind-api, puerto 8080, 1 CPU, 512 MiB, mínimo 0, máximo 3 instancias y timeout de 65 segundos.

En Variables y secretos añade INSTANCE_CONNECTION_NAME, DB_NAME=automind, DB_USER=automind_app, GOOGLE_WEB_CLIENT_ID e INFERENCE_URL con los valores de los pasos anteriores. En Referenciar un secreto selecciona automind-db-password, versión 1, exponer como variable de entorno DB_PASSWORD.

El Dockerfile de API utiliza Python 3.11, instala api/requirements.txt e inicia Uvicorn. Configura APP_ENV=production. No añadas DATABASE_URL en producción: el código utiliza el conector oficial y la configuración de Cloud SQL. Este diseño no usa el montaje /cloudsql ni requiere marcar una conexión Unix en otra pestaña de Cloud Run. Sí necesita la IP pública de la instancia y el rol Cloud SQL Client de la identidad de servicio.

Para la API selecciona Permitir acceso público. Sus rutas seguirán exigiendo el ID token Google. En el motor automind-inference abre Permisos o el panel de información y concede a automind-api@TU_PROYECTO.iam.gserviceaccount.com Cloud Run Invoker (roles/run.invoker). No concedas ese acceso a los Gmail de clientes.

Selecciona la cuenta de compilación automind-build, crea y espera Ready. Si el arranque falla, revisa nombre de conexión, base, usuario, secreto, Cloud SQL Admin API, roles IAM y los archivos SQL ejecutados. El rol de ejecución es automind-api, no automind-build.

Copia la URL HTTPS pública de automind-api a apiUrl en mobile/src/config.ts. Abre esa URL seguida de /v1/catalog en el navegador: debe dar 401 por falta de sesión. La URL privada sin IAM debe ser rechazada. La prueba de usuario real se realiza en el paso 11.

10. Editar catálogo, accesos y solicitudes

Administra datos desde Cloud SQL Studio, entrando a automind como postgres u otro administrador autorizado. No necesitas dar credenciales SQL a los usuarios móviles. Para revisar el catálogo y cambiar un texto:

```
SELECT id,nombre,area,archivo,activo FROM public.modelos ORDER BY id;
UPDATE public.modelos SET descripcion='Suma los tres inputs.'
WHERE id=10000002;
```

Para ocultar y detener un modelo usa UPDATE public.modelos SET activo=FALSE WHERE id=10000002; no borres filas con relaciones. El backend comprueba activo al usarlo. Los IDs iniciales y el detalle de sus archivos están en sql/02-catalogo-y-accesos.sql y catalog/models.json.

Para revisar tus permisos:

```
SELECT email,modelo_id,usuario_id,habilitado FROM public.permisos
WHERE email='artemioaday@gmail.com' ORDER BY modelo_id;
```

Para aprobar una solicitud utiliza una transacción como esta, cambiando el Gmail y el ID por los de la solicitud real:

```
BEGIN;
INSERT INTO public.permisos (email,modelo_id,usuario_id,habilitado)
SELECT email,10000004,google_sub,TRUE FROM public.usuarios
WHERE email='cliente@gmail.com'
ON CONFLICT (email,modelo_id) DO UPDATE SET habilitado=TRUE;
UPDATE public.solicitudes SET estado='approved',
  actualizado_en=CURRENT_TIMESTAMP
WHERE modelo_id=10000004 AND usuario_id=(
  SELECT google_sub FROM public.usuarios WHERE email='cliente@gmail.com');
COMMIT;
```

No cambies solo el estado de una solicitud: la autorización depende de permisos. Para revocar un ID usa UPDATE public.permisos SET habilitado=FALSE WHERE email='cliente@gmail.com' AND modelo_id=10000004;. Para suspender todos los modelos, omite la condición modelo_id. No cambies usuario_id para apuntar a otra cuenta: la clave foránea exige coincidencia de subject y Gmail.

11. Registro y pruebas de aceptación

El archivo inicial reserva a artemioaday@gmail.com los IDs 10000001, 10000002, 10000003, 10000007, 10000008, 10000009, 10000013, 10000014, 10000015, 10000019, 10000020 y 10000021. Son 12 modelos, tres por área. usuario_id permanece NULL antes del registro y se completa con el subject del primer login Google verificado.

Configura la app, compila y entra con tu Gmail. Mis modelos debe mostrar esos 12 y la Store los 24 activos. Abre 10000002 e introduce 10, -3 y 0,5. El resultado debe ser 7,5 unidades. El cálculo se ejecuta desde el archivo de ese ID.

Comprueba en Studio que usuarios contiene tu google_sub y email reales y que tus 12 permisos están vinculados. Las solicitudes solo aparecen cuando se envían desde la app. Ninguna solicitud pendiente concede acceso por sí misma.

Prueba con otro Gmail autorizado como Test user en OAuth: debe poder iniciar sesión y consultar Store, pero no calcular sin permisos. Solicita un ID desde esa cuenta, revisa solicitudes y apruébalo con el paso 10. Revócalo y verifica que la siguiente petición se deniega.

El servidor vuelve a leer PostgreSQL antes de llamar al motor y antes de entregar el resultado. Los resultados ya entregados no se pueden retirar. La cuota transaccional es de 30 cálculos por minuto por subject, en las columnas cuota_minuto y cuota_usos de usuarios.

En Cloud Run > Logs revisa fallos. 401: sesión/cliente Web/Gmail; 403: permiso de ID o IAM del motor; 400: inputs o formato ID; 429: cuota; 503: motor. Un 500 requiere revisar SQL, esquema, privilegios y conexión. No registres tokens ni contraseñas.

La API incluye 22 pruebas Python. Las ocho de integración requieren TEST_DATABASE_URL hacia una base PostgreSQL de prueba vacía; las demás se ejecutan con repositorio y verificador inyectados. En esta entrega se probaron las consultas mediante pg8000 y un motor PGlite local de QA. La red de Cloud SQL, OAuth e IAM deben comprobarse en tu proyecto. Consulta docs/VERIFICACION.md.

12. Compilar Android en tu equipo

Instala Node.js 22 LTS actualizado, JDK 17 y Android Studio. En Android Studio > SDK Manager instala Android SDK Platform 36, Build Tools 36.0.0, NDK 27.1.12297006 y CMake 3.22.1. Extrae el proyecto en una ruta corta, por ejemplo C:\dev\AutoMind. Esta sección usa herramientas locales; la configuración de Google Cloud de los pasos anteriores se realiza con interfaz.

Desde la terminal local de la carpeta mobile ejecuta npm ci para instalar las librerías. Abre mobile/android en Android Studio. Selecciona el JDK 17 para Gradle y deja terminar la sincronización. Si no encuentra el SDK, configura su ubicación desde las opciones de Android Studio; Android puede crear el local.properties correspondiente.

Para el SHA-1 de debug abre el panel Gradle > app > Tasks > android > signingReport y ejecútalo. Si no se muestra esa tarea, desde la terminal local de mobile/android ejecuta .\gradlew.bat signingReport. Copia el SHA-1 de la variante debug al cliente OAuth Android del paso 6.

Para desarrollo, desde mobile ejecuta npm start y, en otra terminal, npm run android con un dispositivo conectado o emulador con Google Play Services. El debug utiliza Metro. Completa config.ts antes de probar el login.

Para un APK/AAB release, en Android Studio usa Build > Generate Signed App Bundle / APK > Create new para crear tu clave privada. Anota la ruta, alias y contraseñas y guarda una copia segura. Configura las cuatro propiedades AUTOMIND_STORE_FILE, AUTOMIND_STORE_PASSWORD, AUTOMIND_KEY_ALIAS y AUTOMIND_KEY_PASSWORD en tu archivo personal ~/.gradle/gradle.properties; no las agregues al repositorio.

Desde mobile/android puedes ejecutar .\gradlew.bat assembleRelease para APK y .\gradlew.bat bundleRelease para AAB. Quedan en app/build/outputs/apk/release y app/build/outputs/bundle/release. El release incluye el bundle y no utiliza Metro. Registra su SHA-1. Si usas Play App Signing, añade también el SHA-1 de la clave de firma de Play y completa los formularios de Play Console.

La compilación nativa anterior se detuvo por espacio de disco; esta entrega no afirma haber generado un APK. El detalle de las comprobaciones está en docs/VERIFICACION.md. Usa un equipo con espacio suficiente para SDK, cachés y compilación.

13. Migración, respaldo e iOS

Si antes subiste la API TypeScript al repositorio, sustituye sus fuentes por la carpeta api actual. Elimina del repositorio los antiguos api/src, api/test, package.json, package-lock.json y tsconfig.json de esa API; conserva los archivos TypeScript de mobile. El nuevo Dockerfile utiliza Python y los endpoints no cambian. No se modifica el esquema PostgreSQL por este cambio de lenguaje.

Si aún no configuraste la nube, sigue este manual directamente. Ya no se utilizan las cuatro bases Firestore ni reglas Firebase. El ZIP nuevo sustituye el backend de documentos por PostgreSQL y no requiere instalar Firebase.

Si tienes datos reales en Firestore, consérvalos mientras preparas PostgreSQL. El SQL inicial solo contiene catálogo y tus 12 permisos; no importa automáticamente otros usuarios, accesos o solicitudes. Revisa esos datos y cópialos deliberadamente antes de cambiar la revisión que atiende la app.

Los permisos adicionales pueden reservarse por Gmail con usuario_id NULL y vincularse cuando esa cuenta entre de nuevo. Para conservar solicitudes antiguas debes copiar primero el subject y email del registro Google real a usuarios, después su modelo y estado a solicitudes. No inventes un subject. Compara los registros y prueba ambas cuentas antes de retirar recursos anteriores.

En Cloud SQL > Copias de seguridad puedes crear un respaldo manual antes de cambios. Para volver atrás en código utiliza las revisiones de Cloud Run; una revisión anterior no deshace cambios SQL. En esta entrega no se borran tablas ni se eliminan tus recursos de nube.

Para rotar la contraseña, cambia automind_app en Usuarios de Cloud SQL, agrega una versión del secreto y despliega una revisión de la API referenciando esa nueva versión. Coordina el cambio para evitar que una revisión antigua siga usando la contraseña anterior.

iOS requiere un Mac con Xcode. Desde mobile ejecuta npm ci, bundle install y, dentro de ios, bundle exec pod install. Abre AutoMind.xcworkspace, configura Team, firma, bundle ID com.automind, cliente iOS y URL scheme del paso 6. Ejecuta o archiva desde Xcode. No se ha compilado iOS en este entorno Windows.

14. Archivos y referencias

sql/01-esquema.sql: cuatro tablas, claves foráneas, IDs de ocho dígitos, estados y cuota.

sql/02-catalogo-y-accesos.sql: 24 modelos con ruta archivo y tus 12 permisos. Conserva cambios previos al repetirlo.

sql/03-privilegios-app.sql: rol limitado automind_app. El backend no puede conceder acceso ni modificar los modelos.

inference/modelos/AREA/MODELO.py: las 24 sumas independientes, con MODEL_ID numérico. inference/registry.py: relación fija entre ID y módulo permitido.

api/automind_api/repository.py: consultas SQL parametrizadas. api/automind_api/database.py: conector Python oficial y pool SQLAlchemy. api/automind_api/main.py inicia la API FastAPI. mobile/src/config.ts: URL de API y clientes OAuth públicos, sin secretos.

Referencias oficiales consultadas:

Cloud SQL PostgreSQL: https://docs.cloud.google.com/sql/docs/postgres/create-instance

Cloud SQL Studio: https://docs.cloud.google.com/sql/docs/postgres/manage-data-using-studio

Usuarios y roles SQL: https://docs.cloud.google.com/sql/docs/postgres/create-manage-users

Conector oficial Python: https://github.com/GoogleCloudPlatform/cloud-sql-python-connector

Secret Manager: https://docs.cloud.google.com/secret-manager/docs/create-secret-quickstart

Secretos en Cloud Run: https://docs.cloud.google.com/run/docs/configuring/services/secrets

Repositorio y despliegue: https://docs.cloud.google.com/run/docs/continuous-deployment

Google OAuth: https://react-native-google-signin.github.io/docs/setting-up/get-config-file

Restricciones SQL: https://www.postgresql.org/docs/current/ddl-constraints.html

Los menús pueden aparecer traducidos. Los pasos describen la configuración desde interfaz y el editor de Studio. No son capturas ni prueba de un despliegue ejecutado en tu cuenta. La entrega incluye fuentes y guía; los recursos, secretos y login reales se configuran con este procedimiento.